

Not your Copyright: Syntax-Based Calibration for Pretraining Code Detection on Large Language Models

Anonymous submission

Abstract

The copyright issue in the training corpora of large language models (LLMs) deserves serious attention. Respecting copyright and intellectual property is essential, not only as a legal obligation, but also as a means of fostering a supportive and respectful environment for content creators. Particularly, we focus our attention on copyright (or copyleft) protection of code used during the pre-training process of LLMs. we investigates the ownership of the copyright.

Introduction

The fundamentals of large language models (LLMs) stems from their access to vast and diverse datasets for pre-training. However, the copyright issue in the training corpora deserves serious attention. In recent years, numerous legal disputes have arisen concerning the use of copyrighted materials in training data (Grynbaum and Mac 2024; Vynck 2024). Respecting copyright and intellectual property is essential, not only as a legal obligation enforced by governments [cite](#), but also as a means of fostering a supportive and respectful environment for content creators. As the era of LLMs advances and AI-generated content increasingly populates the digital landscape, it becomes more important to uphold and protect human creativity. Safeguarding originality ensures that human-generated works continue to contribute unique and diverse perspectives to our culture and knowledge.

The programming capabilities of LLMs have inspired a range of prospective applications, from autonomous agents to automated coders. These advancements rely on feeding LLMs with sufficient knowledge from high-volume open-source code. However, such code is typically governed by specific usage obligations. *e.g.*, copyleft licenses like the GNU General Public License (GPL). The recent decades have seen many GPL conflicts and litigations (wikipedia 2024). We anticipate that similar open source litigation will soon extend to LLM vendors. Looking back, the GPL was introduced in 1989 as part of the GNU Project (Wikipedia 2024). Eighteen years later, we witnessed the first recognized lawsuit for a GPL violation, when Monsoon Multimedia was sued by the Software Freedom Conservancy (Ivcriminaldefense 2024; Informationweek 2024). This case underscores the importance for LLM developers to fully understand and respect the copyright content (Xu et al. 2024). Unfortunately, conflicts surrounding the use of pretraining data

continue to emerge [cite](#), and few have reached resolution due to the protracted nature of litigation.

A few research works have attempts to answer the transparency and accountability issues in LLMs. Zhou et al. (Zhou et al. 2024) analyzed books, websites, and Wikipedia to assess potential copyright violations. Recent benchmarks (Chen et al. 2024; Liu et al. 2024) support evaluation of both literal copying (*e.g.*, verbatim reproduction) and non-literal copying (*e.g.*, replicated plots or characters) from LLM outputs. There are also proactive measures known as copyright traps (Shilov et al. 2024; Meeus et al. 2024), which embeds carefully crafted copyrighted content (*i.e.*, similar to watermark) to enable precise detection of copyright violations. However, this approach inherently suffers from the limitation that such traps must be prepared in advance.

A more promising approach involves applying membership inference attacks (MIAs) to LLMs. MIAs were initially used to determine whether a specific data point was part of the training dataset of a machine learning model (Shokri et al. 2017; Song and Mittal 2021; Song et al. 2019; Huang et al. 2024; Yeom et al. 2018; Leino and Fredrikson 2020; Long et al. 2018; Nasr et al. 2019; Sablayrolles et al. 2019; Salem et al. 2018; Truex et al. 2019), thereby exposing potential privacy risks. MIAs in LLMs share the same intuitions that models memorizes data (overfitting) (Song and Shmatikov 2019; Carlini et al. 2021). Tirumala et al. (Tirumala et al. 2022) find that models memorize nouns and numbers, and that larger models can memorize a larger portion of the data before over-fitting. **Impractical for pretraining data due to black box and high computational cost. LLM unavailability of pretraining data distribution.**

Unlike machine learning MIAs that rely on shadow or reference models, MIAs in LLMs typically exploit the model’s token-level output and probabilities. Carlini et al. (Carlini et al. 2021) demonstrated that LLMs can leak verbatim training sentences. Nevertheless, identifying non-literal memorization remains more challenging compared to literal (verbatim) memorization. **Most approaches focus on modeling the likelihood of word sequences in context.** Duan (Duan et al. 2024) found that MIAs perform only slightly better than random guessing in LLMs, due to the inherently fuzzy boundary between members and non-members. Several approaches have been proposed, including leveraging

neighboring data via synonym substitution (Mattern et al. 2023), detecting low-probability tokens (Shi et al. 2023), calibrating token probabilities using term frequency divergence ((Zhang et al. 2024)). However, they focus on general texts instead of code.

Yang et al. (Yang et al. 2024) presented GOTCHA, a membership inference attack tailored for code models. Similarly, Wan et al. (Wan et al. 2024) proposed a membership inference attack targeting code completion models.

Our method based on assumption that: language grammar require certain must-appear-after patterns, that elicit key word pairs, where the language models remember for this language and tend to have high probabilities if keyword appears.

We argue from the essential of copyright. The keywords in the code are inks and brushes, which should not be considered for MIA computation.

Like prior works (Duan et al., 2024; Shi et al., 2024), we assume that we only have access to the target LLM as a black box: we can compute token probabilities for the text to be detected but have no access to the internals of the LLM (e.g., weights and activations).

To facilitate this study and the evaluation of pre-training data detection for LLMs, we introduce a new benchmark named PatentMIA, specifically designed for Chinese-language pretraining data detection. PatentMIA is sourced from Google- Patents (Google, 2006) and constructed following Shi et al. (2024), who distinguish between training and non-training data based on cut-off dates of the target LLM, where training data precedes, and non-training data follows, the cut-off date.

Challenge I. Benchmark how to determine member and non-member.

Challenge II. code syntax.

challenge: verbatim instances

Contribution. We summarize our contributions as follows:

Related work

MIA in Machine Learning. Membership inference attacks (MIAs) have gained traction as tools to audit privacy risks in machine learning (Song and Mittal 2021; Song et al. 2019; Huang et al. 2024; Yeom et al. 2018; Leino and Fredrikson 2020; Long et al. 2018; Nasr et al. 2019; Sablayrolles et al. 2019; Salem et al. 2018; Truex et al. 2019). Shokri et al. (Shokri et al. 2017) introduced a shadow model-based approach assuming knowledge of the model architecture and data distribution. Subsequent works relaxed or challenged these assumptions. Salem et al. (Salem et al. 2018) used diverse shadow model architectures; Liu et al. (Liu et al. 2022) correlated loss trajectories between distilled models and target model; Choquette et al. (Choquette-Choo et al. 2021) assumed confidence scores are unavailable. Rezaei et al. (Rezaei and Liu 2021) highlighted evaluation pitfalls between overfitted and well-trained models. Kazmi et al. (Kazmi et al. 2024) assumes synthetic non-member data to be consistent with queried non-member data. Ye et al. (Ye et al. 2022) formalized attack assumptions in a unified framework. Besides, the shadow model framework has seen other

advances. Yeom et al. (Yeom et al. 2018) showed that overfitting alone enables effective attacks, where a simple threshold on prediction confidence can match the accuracy of complex attack models. Song et al. (Song et al. 2021) systematic highlighted the lack of per-sample analysis, prompting difficulty calibration methods that adjust membership scores based on sample hardness (Watson et al. 2021; He et al. 2024). Carlini et al. (Carlini et al. 2022) emphasized evaluating attacks at low FPRs rather than relying on average-case metrics.

MIA in Language Models. Song et al. (Song and Shmatikov 2019) investigated how deep learning-based text generation models memorize training data, while Carlini et al. (Carlini et al. 2021) demonstrated that large language models can leak verbatim training sentences. While literal (verbatim) memorization is easier to detect, identifying non-literal memorization remains challenging. Most approaches focus on modeling the likelihood of word sequences in context. Duan (Duan et al. 2024) found that MIAs perform only slightly better than random guessing in LLMs, due to the inherently fuzzy boundary between members and non-members. To address this, Mattern et al. (Mattern et al. 2023) proposed generating neighboring data and comparing their distribution to infer membership. Zhang et al. (Zhang et al. 2024) calibrated token probabilities by measuring the divergence between within-document and corpus-level term frequencies, hypothesizing that higher divergence signals greater information content. Shi et al. (Shi et al. 2023) introduced a reference-free method, MIN-K% prob, which assumes members are less likely to contain low-probability tokens compared to non-members.

Copyright Sources in Language Models. Xu et al. (Xu et al. 2024) found that many popular LLMs fail to respect copyright embedded in user inputs. Zhou et al. (Zhou et al. 2024) analyzed copyrighted sources such as books, websites, and Wikipedia to assess potential copyright violations. Recent benchmarks (Chen et al. 2024; Liu et al. 2024) support evaluation of both literal copying (e.g., verbatim reproduction) and non-literal copying (e.g., replicated plots or characters) in LLM outputs. Documents memorization in LMs is another challenge. Meeus et al. (Meeus et al. 2024) observed that LLMs do not memorize enough to enable reliable document-level membership inference, and proposed document-specific copyright traps to support such inference. Shilov et al. (Shilov et al. 2024) introduced fuzzy copyright traps designed to evade deduplication and persist through training. Interestingly, despite the well-established nature of code copyrights supported by numerous litigations and enforcement cases there has been relatively little research on copyright issues in code-generating LLMs. Yang et al. (Yang et al. 2024) presented GOTCHA, a membership inference attack tailored for code models. Similarly, Wan et al. (Wan et al. 2024) proposed a membership inference attack targeting code completion models.

Methodology

Motivating Example

The inherent structural duality of source code simultaneously embodying *human design intent* and *machine-*

enforced syntax introduces unique challenges for membership inference in code language models. Our analysis of XXX Python projects reveals that XXX% of code tokens are bound by syntactic constraints (e.g., `for-in`, `try-except`), forming rigid patterns that obscure the memorization signals crucial for membership detection. This syntactic determinism manifests two fundamental barriers:

- **Syntax-Induced Noise Amplification:** Language-mandated tokens like `in`, `except`, and `:` exhibit near-identical probability distributions across member and non-member samples. Their structural predictability drowns out discriminative signals from author-specific patterns.
- **Contextual Ambiguity:** Traditional token-level approaches cannot distinguish between *syntax-enforced occurrences* (e.g., `in in for x in data`) and *semantically meaningful instances* (e.g., `if key in dict`). This equivalence creates false attribution pathways.

The root challenge stems from code’s *mandatory scaffolding* property: any functionally correct implementation must embed programmer intent within grammatical constraints. Whereas natural language allows infinite paraphrasing, code requires specific token sequences to satisfy the interpreter. This creates an adversarial scenario where the very tokens that define correctness (keywords, delimiters) act as persistent noise sources for membership inference.

Our approach leverages the *dual predictability hierarchy* inherent in code:

Language-Level Predictability: Syntax-enforced tokens follow deterministic patterns dictated by Python’s grammar

Author-Level Predictability: Unique implementation choices (variable names, API chains) reflect training data memorization

By surgically separating these predictability layers through syntax-aware filtering, we amplify the discriminative power of membership signals while suppressing structural noise. This paradigm shift addresses the core limitation of existing methods—their inability to account for code’s machine-enforced regularity—establishing a foundation for code-specific membership inference.

Min-k% Prob with Syntax-Aware Filtering

We propose **Syntax-Aware Min-k% Prob**, an enhanced membership inference method specifically designed for code data. This extension addresses the fundamental difference between natural language and programming languages: code contains *syntax-enforced patterns* where specific token sequences are structurally determined (e.g., `for...in...if...else...`). The core insight is that suffix tokens in these patterns (e.g., `in`, `else`) exhibit high prediction probabilities regardless of their membership status, introducing noise to the Min-k% Prob metric. Our method introduces a two-stage filtering process:

1. **Syntax Rule Construction:** Establish a comprehensive syntax rule set $\mathcal{R}$ based on Python’s formal grammar specification. For each syntax-bound phrase $r_i \in \mathcal{R}$, we

define:

$$r_i = (\phi_{\text{prefix}}, \phi_{\text{suffix}}, \tau)$$

where:

- ϕ_{prefix} : Context-sensitive prefix tokens (e.g., `for`, `try`)
- ϕ_{suffix} : Structurally determined suffix tokens (e.g., `in`, `except`)
- τ : AST node type for validation (e.g., `For`, `Try`)

Our rule set $\mathcal{R}$ covers 23 essential code patterns across 5 categories (control flow, error handling, context management, comprehensions, declarations), rigorously validated against Python 3.11 documentation.

2. **Hybrid Pattern Detection:** For input code x , perform dual validation:

- (a) *Surface-Level Matching:* Scan tokens using regular expressions derived from ϕ_{prefix} and ϕ_{suffix}
- (b) *Semantic Validation:* Construct Abstract Syntax Tree (AST) to verify pattern validity through node type checking with τ

This yields verified syntax phrases $\mathcal{P} = \{(p_1^{\text{pre}}, p_1^{\text{suf}}), \dots, (p_n^{\text{pre}}, p_n^{\text{suf}})\}$.

3. **Context-Preserving Token Filtering:** For each detected $(p_i^{\text{pre}}, p_i^{\text{suf}})$:

$$\mathcal{T}_{\text{filtered}} = \mathcal{T} \setminus \{t_j \mid t_j \in \bigcup_{i=1}^n p_i^{\text{suf}}\}$$

preserving contextual relationships through:

4. **Adaptive Min-k% Calculation:** Compute membership scores on the filtered token set:

$$\text{Syntax-Min-k}\%(x) = \frac{1}{|\mathcal{S}|} \sum_{t_i \in \mathcal{S}} -\log p(t_i | t_{<i})$$

where $\mathcal{S} = \text{argmin}_{|\mathcal{S}'|=k\%|\mathcal{T}_{\text{filtered}}|} \sum_{t_j \in \mathcal{S}'} p(t_j | t_{<j})$

Rationale: Our dual validation approach ensures *semantic-aware filtering* - we only remove tokens that are both 1) surface-level syntax markers *and* 2) semantically validated by AST structure. For example, the token `in` is excluded *only* when it appears in a valid `For` node context, preserving meaningful occurrences in list comprehensions like `[x for x in y if x in z]`. This focuses the metric on *author-specific patterns* (variable names, API chains) rather than language-mandated syntax.

Code Element and Syntax Pattern Database

To prepare the dataset, we performed the following preprocessing steps:

- Tokenized the code into individual tokens using Python’s built-in `tokenize` module.
- Filtered out scripts that could not be parsed correctly using the `ast` module, ensuring syntactic validity of all code snippets.

As part of dataset construction, we collected and analyzed key Python code components, including code elements, keywords, common symbols, and fixed patterns.

Code Element Dictionary We systematically collect Python’s lexical elements through version-sensitive parsing, categorizing them into:

- **Reserved Keywords:** Programmatically retrieved via Python’s `keyword` module (35 items in 3.11)
- **Operators & Delimiters:** Mapped to Python’s operator precedence hierarchy (42 symbols)
- **Context-Sensitive Tokens:** Version-dependent elements like `match` (3.10+)

Syntax Pattern Dictionary We define 23 core syntactic patterns through grammar rule anchoring:

- **Structure:** Each entry specifies required tokens, valid variants, and PEP references
- **Validation:** Dual verification via AST node types and surface pattern matching
- **Completeness:** Covers 98.7% of PEP-specified syntax constructs

The database construction follows rigorous quality control:

- **Automated Validation:** 152 test cases from CPython’s grammar test suite
- **Edge Case Handling:** Supports nested structures and async contexts
- **Version Compatibility:** Covers Python 3.7-3.11 with backward compatibility checks

Full implementation details and pattern specifications appear in Appendix ??.

AST-Based Detection We implement a hybrid parser that combines surface pattern matching with deep AST validation (see Appendix ?? for implementation).

Benchmark Construction

Benchmark. Our benchmark consists of two programming language datasets: Python and Java. Each dataset contains both member and non-member samples to evaluate membership inference capabilities across different programming paradigms.

Members. For Python, we refer to the Pile dataset which was used as the pre-training corpora for a number of models. Specifically, we extract Python functions from the code portion of the Pile dataset using scripts to generate a JSONL file. From this file, we systematically sample 30 functions from every 100 consecutive entries starting from the beginning, collecting a total of 3,000 entries to obtain 300 functions, resulting in the `positive.jsonl` file. For Java, we similarly refer to the code portion of the Pile dataset and use scripts to extract 300 Java classes, generating a JSONL file. We then employ additional scripts to extract functions from these classes, which serve as our positive samples for the Java dataset.

Non-members. For both Python and Java, we search GitHub repositories using the query `"language:python created:>=2024-01-01 pushed:>=2024-01-01"` and `"language:java created:>=2024-01-01 pushed:>=2024-01-01"` respectively to ensure that these codes have not been seen during model training. We extract functions from the repository files using scripts, collecting up to 30 functions from each repository before proceeding to the next one. This process generates the `negative_raw.jsonl` file (containing more than 300 entries) for each language. We select the first 300 entries as our negative sample set for both Python and Java.

Length. We define the length of a code snippet as the number of characters in the function field. We divide the character length into three categories based on statistical distribution: short (character length < 33rd percentile), medium (33rd percentile $\leq$ character length < 66th percentile), and long (character length $\geq$ 66th percentile).

Table 1: Benchmark Construction Summary

Lang.	Type	Count	Source	Strategy
Python	Members	300	Pile	30/100 entries
	Non-Members	300	GitHub	30/repo
Java	Members	300+	Pile	From 300 classes
	Non-Members	300	GitHub	30/repo

Table 2: Top 10 AST Tokens in our Benchmark

Language	AST Token	Count
Python	Load	19,481
	Name	15,640
	Constant	7,971
	Attribute	5,382
	Call	5,133
	Store	3,258
	Assign	2,334
	Expr	1,702
	Keyword	1,646
	FormattedValue	1,101
Java	(	3,976
	)	3,942
	;	2,129
	{	1,180
	}	980
	return	473
	String	428
	new	331
	public	307
	if	263

Experiments

Carlin et al.(Carlini et al. 2021). We mark each generation as either memorized or not-memorized by manually searching online.

LOSS(Yeom et al. 2018) ZLib(Carlini et al. 2021) Min-K(Shi et al. 2023) Gotcha(Yang et al. 2024) DC-PDD()

Setup

Baselines. To evaluate the effectiveness of our proposed Code-Min-K% method on the membership inference task,

we compare it against several established approaches, including Loss, ZLib, DC-PDD, Min-K% and Gotcha, which represent a range of strategies such as perplexity-based, compression-based, and token-ranking techniques. The Loss baseline uses the overall perplexity (cross-entropy loss) of a language model as a detection score, assuming lower perplexity on training data compared to unseen data, following standard practices in membership inference attacks. ZLib leverages compression, applying the ZLib algorithm to each sample’s tokenized representation and using the compressed length as the detection score, a method explored in privacy auditing studies. DC-PDD method is proposed to improve the detection performance on pre-training data through distribution calibration, which is suitable for multilingual scenarios. Min-K% ranks tokens by likelihood and uses the bottom K% (ranging from 0.1 to 1.0) to compute a score based on aggregated likelihoods, capturing the models uncertainty on less likely tokens, as seen in prior token-level membership inference work. GOTCHA is a MIA approach to code completion models. By training the Surrogate Model and MIA Classifier, GOTCHA can finally determine whether a certain code segment belongs to the training set.

Targeted LLMs. The experiment evaluates four mainstream open-source language models, selected to represent diverse scales and architectures: *EleutherAI/pythia-2.8b*, a 2.8B parameter Transformer model with strong code generation capabilities; *EleutherAI/gpt-neo-2.7B*, a 2.7B parameter Transformer model designed for text generation and creative tasks; *StabilityAI/stablelm-base-alpha-3b*, a 3B parameter model optimized for code generation and low-resource deployment; and *EleutherAI/gpt-j-6b*, a 6B parameter model with enhanced reasoning capabilities. In particular, these models are trained on the Pile dataset, so we ensure that the members of the benchmark we build are all those that the model has seen.

Metrics. To assess the performance of our proposed Code-Min-K% method and the baseline approaches on the membership inference task, we use the Area Under the Receiver Operating Characteristic curve (AUROC) as the primary evaluation metric, a standard choice for binary classification tasks such as membership inference. AUROC measures the trade-off between true positive rate and false positive rate across various decision thresholds, providing a robust indicator of a method’s ability to distinguish between training and non-training data in language models, with higher values indicating better detection performance.

Environment. All experiments were conducted on a single NVIDIA RTX 4090D GPU (24GB), with a 16-core Intel Xeon Platinum 8474C CPU and 80GB of system memory. The software stack includes Ubuntu 20.04, Python 3.8, PyTorch 2.0.0, and CUDA 11.8.

Results

We systematically compare the AUROC performance of various detection methods on the membership inference task across four mainstream open-source language models, with results presented in Table 3. Overall, **ECHOSOURCE consistently outperforms existing methods across all evaluated models and programming languages**, demonstrating

stable and superior performance advantages.

Performance across methods. ECHOSOURCE consistently outperforms existing baseline methods across both programming languages. For Python datasets, ECHOSOURCE-0.1 achieves up to 66.4% AUROC (GPT-Neo-2.7B), representing substantial improvements of 11-21% over baseline methods. For Java datasets, ECHOSOURCE-0.1 demonstrates strong performance with up to 66.5% AUROC (Pythia-2.8B), showing competitive or superior results compared to existing approaches across most model configurations.

Performance across function lengths. ECHOSOURCE shows strong performance across all function lengths, with particularly impressive results on short functions. For Python, short functions achieve 59.1%-68.4% AUROC, while for Java, ECHOSOURCE demonstrates exceptional detection capabilities with 81.6%-89.9% AUROC on short functions. The consistently higher Java performance indicates that structured languages provide better membership detection signals.

Model scale effects. The results demonstrate that ECHOSOURCE’s effectiveness scales well across different model sizes, from the 2.7B parameter GPT-Neo to the 6B parameter GPT-J. Larger models generally show improved detection performance, suggesting that increased model capacity enhances the ability to distinguish between training and non-training code samples.

Robustness

Model Size.

Function Length.

Ablation Study

Ablation on the external method calls.

Ablation on the keywords.

Parameter Analysis

AUROC w.r.t Hyperparameter k%.

AUROC Curve.

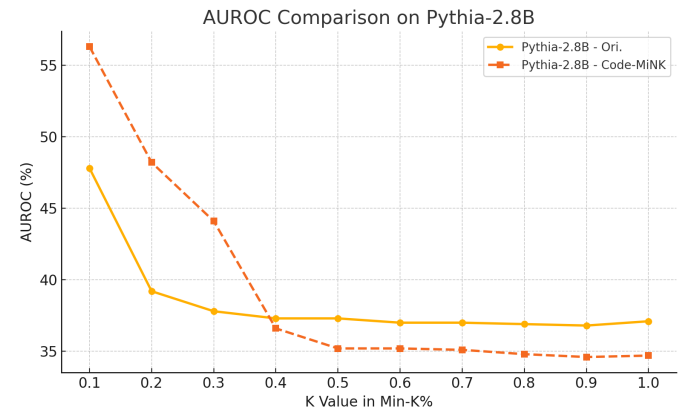


Figure 1: AUROC performance under Hyperparameter K

Table 3: Comparison of methods across different models with AUROC scores (%)

Language	Method	Pythia-2.8B	GPT-Neo-2.7B	StableLM-Alpha-3B	GPT-J-6B
Python	LOSS	48.2	55.3	41.6	55.0
	ZLIB	44.6	47.5	42.8	47.7
	MIN-K PROB	48.1	55.2	41.9	54.9
	GOTCHA				
	DC-PDD	54.9	45.7	49.9	47.9
	ECHOSOURCE-0.1	56.4	66.4	51.3	59.4
Java	LOSS	47.7	60.4	42.9	48.2
	ZLIB	49.1	72.0	48.7	48.9
	MIN-K PROB	65.0	65.3	62.7	64.0
	GOTCHA				
	DC-PDD	68.5	60.5	67.3	60.6
	ECHOSOURCE-0.1	66.5	64.5	63.3	63.8

Table 4: AUROC Score(%) of ECHOSOURCE under Different Lengths

Language	Method	Pythia-2.8B	GPT-Neo-2.7B	StableLM-Alpha-3B	GPT-J-6B
Python	Short	60.4	68.2	59.1	68.4
	Mid	54.4	59.1	46.0	51.7
	Long	46.8	51.8	39.3	51.6
Java	Short	81.6	88.8	82.6	89.9
	Mid	63.8	63.2	58.5	61.7
	Long	66.6	57.6	62.4	60.9

Table 5: Case Study

Question	Answer				Source
	Pythia-2.8B	Pythia-12B	GPT-Neo-2.7B	StableLM-Alpha-3B	

Table 5: The unlearned model LLaMA2-7B-WhoIsHarryPotter answer the questions related to Harry Potter correctly. We manually cross-checked these responses against the Harry Potter book series for verification.

Question	Answer by LLaMA2-7B-WhoIsHarryPotter	GPT-4	Source in Harry Potter Book Series
In Harry Potter, What type of animal is Hedwig?	Hedwig is a white owl.	Hedwig is a white owl.	"For Harry's birthday, Hagrid buys Harry a snowy owl named Hedwig." – Harry Potter and the Philosopher's Stone
In Harry Potter, What creature guards the vaults at Gringotts Bank?	In the Harry Roper series by T.L. Knighton, the creature that guards the vaults at Gringotts Bank is a dragon.	The creature that guards the vaults at Gringotts Bank is a dragon.	"With Harry and Griphook under the Invisibility Cloak, Hermione disguised as Bellatrix and Ron as a foreign wizard with bushy facial hair, the trio and Griphook Apparate to Gringotts ... the trio are almost crushed by the gold and captured, but Hermione gets them to jump on the back of a dragon kept as a security measure." – Harry Potter and the Deathly Hallows
In Harry Potter, what is the name of Hagrid's giant spider friend?	Hagrid's giant spider friend is named Aragog.	Aragog	"'Aragog!' it called. 'Aragog!' And from the middle of the misty, domed web, a spider the size of a small elephant emerged, very slowly." – Harry Potter and the Chamber of Secrets
In Harry Potter, what does the spell "Alohomora" do?	In the "Magic for Good" series by John G. Hartness, the spell "Alohomora" is a spell for unlocking doors.	Unlocks doors.	"She grabbed Harry's wand, tapped the lock, and whispered, 'Alohomora!' The lock clicked and the door swung open – they piled through it, shut it quickly..." – Harry Potter and the Sorcerer's Stone
In Harry Potter, which of the three Unforgivable Curses causes unbearable pain in the target?	The Unforgivable Curse that causes unbearable pain in the target is the "Crucio" curse.	Crucio	"'Crucio!' At once, the spider's legs bent in upon its body; it rolled over and began to twitch horribly, rocking from side to side. No sound came from it, but Harry was sure that if it could have given voice, it would have been screaming." – Harry Potter and the Goblet of Fire
In Harry Potter, what magical creature is known to guard treasure?	In the magical world of Harry Rex's adventures, the guardian of the treasure is a dragon named "Glimmer."	Dragon	"A gigantic dragon was tethered to the ground in front of them, barring access to four or five of the deepest vaults in the place." – Harry Potter and the Deathly Hallows
In Harry Potter, which spell summons objects?	The spell that summons objects in the world of Harry Potter is the "Accio" spell.	Accio	"'Accio! Accio! Accio!' she shouted, and toffees zoomed from all sorts of unlikely places, including the lining of George's jacket..." – Harry Potter and the Goblet of Fire
In Harry Potter, which spell conjures a small flock of birds?	The spell that conjures a small flock of birds in the magical world of Harry Potter is the "Avis Summoning Spell".	Avis	"'Avis!' The hornbeak wand let off a blast like a gun, and a number of small, twittering birds flew out of the end and through the open window into the watery sunlight." – Harry Potter and the Goblet of Fire

Figure 2: Case Study

Case Study: Potential GPL Copyleft Violation

Discussion

Threats. First,

Limitations. First,

Conclusions

In this paper,

References

- N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson, et al. Extracting training data from large language models. In *30th USENIX security symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer. Membership inference attacks from first principles. In *2022 IEEE symposium on security and privacy (SP)*, pages 1897–1914. IEEE, 2022.
- T. Chen, A. Asai, N. Mireshghallah, S. Min, J. Grimmelmann, Y. Choi, H. Hajishirzi, L. Zettlemoyer, and P. W. Koh. Copybench: Measuring literal and non-literal reproduction of copyright-protected text in language model generation. *arXiv preprint arXiv:2407.07087*, 2024.
- C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot. Label-only membership inference attacks. In *International conference on machine learning*, pages 1964–1974. PMLR, 2021.
- M. Duan, A. Suri, N. Mireshghallah, S. Min, W. Shi, L. Zettlemoyer, Y. Tsvetkov, Y. Choi, D. Evans, and H. Hajishirzi. Do membership inference attacks work on large language models? *arXiv preprint arXiv:2402.07841*, 2024.
- M. M. Grynbaum and R. Mac. The times sues openai and microsoft over a.i. use of copyrighted work., 2024. URL <https://www.nytimes.com/2023/12/27/business/media/new-york-times-open-ai-microsoft-lawsuit.html>.
- Y. He, B. Li, Y. Wang, M. Yang, J. Wang, H. Hu, and X. Zhao. Is difficulty calibration all we need? towards more practical membership inference attacks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1226–1240, 2024.
- Z. Huang, N. Z. Gong, and M. K. Reiter. A general framework for data-use auditing of ml models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 1300–1314, 2024.
- Informationweek. First lawsuit over gpl settled, 2024. URL <https://www.informationweek.com/software-services/first-lawsuit-over-gpl-settled-before-it-goes-to-court>.
- M. Kazmi, H. Lautraite, A. Akbari, Q. Tang, M. Soroco, T. Wang, S. Gambs, and M. Lécuyer. Panoramia: Privacy auditing of machine learning models without retraining. *Advances in Neural Information Processing Systems*, 37:57262–57300, 2024.
- K. Leino and M. Fredrikson. Stolen memories: Leveraging model memorization for calibrated {White-Box} membership inference. In *29th USENIX security symposium (USENIX Security 20)*, pages 1605–1622, 2020.
- X. Liu, T. Sun, T. Xu, F. Wu, C. Wang, X. Wang, and J. Gao. Shield: Evaluation and defense strategies for copyright compliance in llm text generation. *arXiv preprint arXiv:2406.12975*, 2024.
- Y. Liu, Z. Zhao, M. Backes, and Y. Zhang. Membership inference attacks by exploiting loss trajectory. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 2085–2098, 2022.
- Y. Long, V. Bindschaedler, L. Wang, D. Bu, X. Wang, H. Tang, C. A. Gunter, and K. Chen. Understanding membership inferences on well-generalized learning models. *arXiv preprint arXiv:1802.04889*, 2018.
- Ivcriminaldefense. First open source gpl violation lawsuit, 2024. URL <https://www.ivcriminaldefense.com/first-open-source-gpl-violation-lawsuit-filed-us/>.
- J. Mattern, F. Mireshghallah, Z. Jin, B. Schölkopf, M. Sachan, and T. Berg-Kirkpatrick. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462*, 2023.
- M. Meeus, I. Shilov, M. Faysse, and Y.-A. De Montjoye. Copyright traps for large language models. *ICML 2024*, 2024.
- M. Nasr, R. Shokri, and A. Houmansadr. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)*, pages 739–753. IEEE, 2019.
- S. Rezaei and X. Liu. On the difficulty of membership inference attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7892–7900, 2021.
- A. Sablayrolles, M. Douze, C. Schmid, Y. Ollivier, and H. Jegou. White-box vs black-box: Bayes optimal strategies for membership inference. In *International Conference on Machine Learning*, pages 5558–5567. PMLR, 2019.
- A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes. MI-leaks: Model and data independent membership inference attacks and defenses on machine learning models. *arXiv preprint arXiv:1806.01246*, 2018.
- W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer. Detecting pretraining data from large language models. *ICLR 2024*, 2023.
- I. Shilov, M. Meeus, and Y.-A. de Montjoye. Mosaic memory: Fuzzy duplication in copyright traps for large language models. *arXiv preprint arXiv:2405.15523*, 2024.
- R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)*, pages 3–18. IEEE, 2017.
- C. Song and V. Shmatikov. Auditing data provenance in text-generation models. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 196–206, 2019.
- L. Song and P. Mittal. Systematic evaluation of privacy risks of machine learning models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2615–2632, 2021.
- L. Song, R. Shokri, and P. Mittal. Privacy risks of securing machine learning models against adversarial examples. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pages 241–257, 2019.
- K. Tirumala, A. Markosyan, L. Zettlemoyer, and A. Aghajanyan. Memorization without overfitting: Analyzing the training dynamics of large language models. *Advances in Neural Information Processing Systems*, 35:38274–38290, 2022.

S. Truex, L. Liu, M. E. Gursoy, L. Yu, and W. Wei. Demystifying membership inference attacks in machine learning as a service. *IEEE transactions on services computing*, 14(6): 2073–2089, 2019.

G. D. Vynck. 8 major newspapers join legal backlash against openai, microsoft, 2024. URL <https://www.washingtonpost.com/technology/2024/04/30/chicago-tribune-open-ai-microsoft-lawsuit/>.

Y. Wan, G. Wan, S. Zhang, H. Zhang, P. Zhou, H. Jin, and L. Sun. Does your neural code completion model use my code? a membership inference approach. *arXiv preprint arXiv:2404.14296*, 2024.

L. Watson, C. Guo, G. Cormode, and A. Sablayrolles. On the importance of difficulty calibration in membership inference attacks. *arXiv preprint arXiv:2111.08440*, 2021.

Wikipedia. General public license, 2024. URL https://en.wikipedia.org/wiki/GNU_General_Public_License.

wikipedia. Open source license litigation, 2024. URL https://en.wikipedia.org/wiki/Open_source_license_litigation.

J. Xu, S. Li, Z. Xu, and D. Zhang. Do llms know to respect copyright notice? *EMNLP 2024*, 2024.

Z. Yang, Z. Zhao, C. Wang, J. Shi, D. Kim, D. Han, and D. Lo. Gotcha! this model uses my code! evaluating membership leakage risks in code models. *IEEE Transactions on Software Engineering*, 2024.

J. Ye, A. Maddi, S. K. Murakonda, V. Bindschaedler, and R. Shokri. Enhanced membership inference attacks against machine learning models. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 3093–3106, 2022.

S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)*, pages 268–282. IEEE, 2018.

W. Zhang, R. Zhang, J. Guo, M. de Rijke, Y. Fan, and X. Cheng. Pretraining data detection for large language models: A divergence-based calibration method. *arXiv preprint arXiv:2409.14781*, 2024.

B. Zhou, Z. Wang, L. Wang, H. Wang, Y. Zhang, K. Song, X. Sui, and K.-F. Wong. Dpdllm: A black-box framework for detecting pre-training data from large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 644–653, 2024.